

Proving the hard-to-round search

From `htr.c` with MPFR to a search with a proved exponential

Names set in `monospace` are identifiers of `code/APaul/htr.c`, of the C files in `code/APaul/htrplan/`, or of the Rocq files in `code/APaul/rocq/`. Every number is marked **read** (off the code or a file), **measured** (a run), or **computed** (arithmetic on the other two).

1 The goal

Let x be a double and e the exponent of $\exp x$, so that $Y(x) = \exp(x) \cdot 2^{54-e}$ lies in $[2^{53}, 2^{54})$. The integers are the **round-bit grid**: the doubles and the midpoints between them. We say that x is **hard to round** at level m when

$$\text{dist}(Y(x), \mathbb{Z}) < 2^{-m},$$

that is, when the m bits after the round bit of $\exp x$ are all equal.

The theorem we want. For an interval $[x_0, x_1)$, the list the program prints contains every double of $[x_0, x_1)$ that is hard to round at level m . We ask for a **superset**: the program may print a false case, it must not miss a true one. The program works in two stages, a search that produces **candidates** and a check that decides each candidate; the theorem covers both.

The final aim is the whole range where `exp` gives neither an infinity nor a NaN, about $x \in [-745, 709.78]$. Below $|x| < 2^{-54}$ the answer is known by hand (`exp x` rounds to 1 or just below). What is left is 64 binades on each side, $2 \cdot 64 \cdot 2^{52} = 2^{59} \approx 5.8 \cdot 10^{17}$ doubles (**computed**).

2 How the search works

The search cuts $[x_0, x_1)$ into **chunks** of $n = 2^{20}$ consecutive doubles (**read**, **search**). On a chunk starting at x , with u the ulp of x ,

$$Y(x + iu) \approx Y(x) + i \cdot Y(x)u, \quad 0 \leq i < n,$$

because the derivative of `exp` is `exp`. Only the fractional part matters, kept on 64 bits:

- **A** approximates the fractional part of $Y(x)$, times 2^{64} ;
- **B** approximates the fractional part of the slope $Y(x)u$, times 2^{64} ;
- **E** is a window: 2^{64-m} , the target, plus a bound on how far the straight line drifts from the curve over the chunk, $Y(x)u^2n^2/2$.

The inner loop adds **B** to **A** n times and reports every i where **A** is within **E** of an integer (the test $\mathbf{A} < 2 \cdot \mathbf{E}$ after shifting by **E**). This loop, a comparison and an addition, is the whole cost: 99.99% of the run on $[0.25, 0.25001)$ (**measured**, the Capla benchmark).

The one idea of the proof. The true value and the line differ by at most (drift + errors of **A**, **B**). If the window is at least target + drift + errors, no hard case falls outside it.

3 Three programs

- **Program 1, the initial program**, is `htr.c` as it is, with three calls to MPFR. It is not proved; it is the reference the others are tested against. It is shown in Section A.
- **Program 2, the generic search**, is `htr.c`'s search with the calls to MPFR abstracted into parameters, and a contract on them. The proof starts here.
- **Program 3, the proved evaluator**, is an exponential in integer arithmetic that meets the contract, with no MPFR. Plugged into Program 2, it gives a search with nothing trusted but the arithmetic.

All the C is in `code/APaul/htrplan/`, and every listing below is read from it.

4 Program 2: the generic search

Program 2 is `htr.c` with its three MPFR functions turned into parameters: `dd_exp` becomes `eval`, `ref_exp` becomes `expo`, and `check` becomes `report`. Its interface, with the contract on the parameters, is `search.h`:

```
// Program 2, the generic search: htr.c with the evaluator as a parameter.

#include <stdint.h>

// eval (&h, &l, &s, x) approximates exp(x) by h + l + s. Let e be the
// exponent of exp(x) (as frexp gives it). The contract is:
// (C1) |h + l + s - exp(x)| <= 2^-kappa * exp(x)
// (C2) |l| < 2^(e-53) and |s| < 2^(e-54)
// C2 is what the search needs to read the bits after the round bit from
// l and s alone. kappa is what the proof of the window asks for.
typedef void (*eval_t) (double *h, double *l, double *s, double x);

// expo (x) returns e, the exponent of exp(x), exactly.
typedef int (*expo_t) (double x);

// report (x) is called on every candidate. The theorem to prove is that
// every double of [x0, x1) that is hard to round at level m is reported.
typedef void (*report_t) (double x, int m);

void search (eval_t eval, expo_t expo, report_t report,
             double x0, double x1, int m);
```

The search itself, `search.c`, is `htr.c`'s `search` line for line, except for the three names:

```
// Program 2, the generic search: the search of htr.c, line for line,
// with dd_exp replaced by eval, ref_exp by expo, and check by report.

#include <assert.h>
#include <math.h>
#include "search.h"

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m identical bits after round bit
void
search (eval_t eval, expo_t expo, report_t report,
        double x0, double x1, int m)
```

```

{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

    // check ulp(x0) = ulp(nextbelow(x1))
    frexp (x0, &e0);
    frexp (nextafter (x1, x0), &e1);
    assert (e0 == e1);

    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
    e = expo (x0);
    e1 = expo (nextafter (x1, x0));
    assert (e == e1);

    double h, l, s;
    double x = x0;
    while (x < x1) {
        eval (&h, &l, &s, x);
        double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
        double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

        // prepare loop
        // A is the bits of h+l after the round bit
        h = ldexp (h, 54 - e); // now ulp(h) = 2
        l = ldexp (l, 54 - e);
        // we now have -2 < l < 2
        if (l >= 1.0) l -= 1.0;
        while (l < 0) l += 1.0;
        assert (0 <= l && l < 1);
        s = ldexp (s, 54 - e);
        uint64_t lu = get_uint64 (l), su = get_uint64 (s);
        uint64_t A = lu + su;
        // scale the derivatives too
        dh = ldexp (dh, 54 - e);
        dl = ldexp (dl, 54 - e);
        dd = ldexp (dd, 54 - e);
        uint64_t B = get_uint64 (dh) + get_uint64 (dl);
        // the maximal error is bounded by dd/2*n^2
        dd = dd / 2.0 * n * n;
        // hard-to-round cases are at distance < 2^-(m+1) ulp(h)
        uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
        // we want to find values of A in [-E, E]
        A += E; // shift A
        // now we want to find values of A in [0, 2*E]
        for (uint64_t i = 0; i < n; i++) {
            if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
                double xi = x + i * ux;
                report (xi, m);
            }
            A += B;
        }
        x += n * ux;
        if (x > x1)
            n = (x1 - x) / ux;
    }
}

```

```

}
}

```

4.1 The instantiation that gives back Program 1

Take for the parameters `htr.c`'s own three functions, in `eval_mpfr.c`:

parameter	instance
eval	eval_mpfr = htr.c's dd_exp, unchanged
expo	expo_mpfr = htr.c's ref_exp, returning the exponent of its result instead of the result
report	check_mpfr = htr.c's check, returning its verdict; the caller (<code>main.c</code>) prints the hard cases

```

// The evaluator of the initial program (htr.c): exp with MPFR, trusted.
// Also check, which decides a candidate; it is not part of the superset.

#include <stdio.h>
#include <mpfr.h>
#include "eval.h"

// the exponent of exp(x): htr.c's ref_exp rounds toward zero, so that the
// result never rounds up to the next power of 2
int
expo_mpfr (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    int e;
    frexp (mpfr_get_d (y, MPFR_RNDN), &e);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return e;
}

// htr.c's dd_exp: exp(x) at 161 bits, split into three doubles
void
eval_mpfr (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

```

```

}

// htr.c's check: is x hard to round at level m
int
check_mpfr (double x, int m)
{
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
    mpfr_set_d (y, x, MPFR_RNDN);
    mpfr_exp (z, y, MPFR_RNDN);
    mpfr_set (y, z, MPFR_RNDN);
    int hard = mpfr_cmp (y, z) == 0;
    mpfr_clear (y);
    mpfr_clear (z);
    return hard;
}

```

With them, Program 2 is Program 1 again: the test prints the same candidates and the same hard cases as `htr.c` itself, line for line (Section 6).

4.2 The obligations on the exp evaluator

These are what any instance must prove; nothing here depends on how it is implemented.

#	obligation
E1	(C1) accuracy: $ h + l + s - \exp x \leq 2^{-\kappa} \exp x$
E2	(C2) shape: $ l < 2^{e-53}$ and $ s < 2^{e-54}$, where e is the exponent of $\exp x$. After scaling, l must be in $(-2, 2)$ and s in $(-1, 1)$ (read , <code>htr.c</code> lines 110 and 115)
E3	expo returns the exponent of $\exp x$ exactly. It is only used to check that exp stays in one binade on $[x_0, x_1)$. <code>htr.c</code> rounds toward zero so that the result never rounds up to the next power of 2
E4	the check called from report never rejects a hard case. It may accept a false one, or answer “undecided”, which is kept

E4 is there because the theorem is about the printed list, not only the candidates. If one only wanted the candidates, E4 would not be needed.

How large must κ be. An error of $2^{-\kappa}$ relative is about $2^{54-\kappa}$ on the grid, so it is below one unit of **A** (i.e. 2^{-64}) when $\kappa \geq 118$ (**computed**). With $\kappa \approx 120$, the evaluator errors are small next to the drift, which is about 2^{48} units on $[0.25, 0.25001)$ (**computed** from `htr.c`’s formula). MPFR at 161 bits gives $\kappa \approx 158$, much more than needed.

4.3 The obligations on the code of Program 2

These hold for any **eval**, **expo** and **report** that meet E1 to E4.

#	obligation	state
2a	the conversions to 64 bits: ldexp , get_uint64 , the reduction of l into $[0, 1)$, the wrap modulo 2^{64}	to do, fiddly
2b	the window covers target + drift + the errors of E1 and 2a	to do, the hard point
2c	the inner loop reports exactly the i in the window	done: Scan.v
2d	the chunks cover $[x_0, x_1)$	done at the nat layer: ScanAll.v

On 2b. E is target + drift and nothing else (**read**). Three errors are not in it: the curvature is taken at the start of the chunk and not its maximum over the chunk; the two truncations of B add up over n steps; and A itself is truncated. Our notes of 2026-08-01 say they fit inside the margin; this has not been re-checked. The proof decides it. If they do not fit, the fix is one term added to E .

What the code does not handle. Being `htr.c`'s search, Program 2 asserts things that hold on $[0.25, 0.5)$ but not on the whole range (**read**):

- `get_uint64` asserts its argument is in $(-1, 1)$. The slope $Y(x)u$ is in $[2^{e_0}, 2^{e_0+1})$ where $x \in [2^{e_0-1}, 2^{e_0})$, so it passes only for $|x| < 1/2$ (**computed**). Only the fractional part of the slope matters, so the fix is to drop the integer part first.
- $[x_0, x_1)$ must sit in one binade of x and of $\exp x$ (asserted). The full range must be cut accordingly: about $2 \cdot 64$ binades of x and a cut every $\ln 2$ for $\exp x$ (**computed**).
- Near the ends, $\exp x$ becomes subnormal (below $x \approx -708$) or overflows (above $x \approx 709.78$). The definition of “hard to round” changes for subnormal outputs.
- The last chunk overruns x_1 ; the line `n = (x1 - x) / ux` is never used. This is harmless for a superset of $[x_0, x_1)$.

5 Program 3: another instantiation of Program 2

Program 3 is Program 2 with other parameters: an exponential in integer arithmetic, with no MPFR. The proof of Program 2 (2a to 2d) is used as it is. What is left is to prove E1 to E4 for the new parameters.

parameter	instance
eval	eval_fix: $\exp x$ as a fixed-point integer y , split into three doubles
expo	expo_fix: the exponent of y
report	check_fix: decides a candidate from the same y ; the caller prints the hard and the undecided cases

5.1 The candidate evaluator

How fast must it be? Not very. `eval` is called once per chunk. The inner loop runs about $1.5 \cdot 10^9$ steps a second (**measured**, one P-core, turbo off), so a chunk takes about 0.7 ms (**computed**). An evaluator of $10 \mu s$ would add 1.4% (**computed**). On the full range there are $2^{59}/2^{20} = 2^{39} \approx 5.5 \cdot 10^{11}$ chunks (**computed**), and the whole search is about $3.9 \cdot 10^8$ core-seconds, 12 core-years (**computed**), whatever the evaluator. So the evaluator is chosen for the **proof effort**, not for speed.

Three possible evaluators.

evaluator	how	certificates	difficulty
Tang, fixed point	$x = k \ln 2 + j/256 + r$; a table of 256 values $\exp(j/256)$; one polynomial for $\exp r$, $ r \leq 2^{-9.5}$: degree 10 gives 2^{-130} , degree 13 gives 2^{-169} (computed from the Taylor bound); integers only	1 polynomial, 256 table entries, 1 reduction	3
pieces + section 5	one certified polynomial per piece, like <code>Cheb.v</code> ; the chunk seeds by exact additions (<code>Shift.v</code> , <code>Pdir_exp</code>)	$6 \cdot 10^5$ at degree 7, $4 \cdot 10^3$ at degree 15, for 2^{-100} (computed , formula checked at one point)	2 each, but volume

short Taylor near 0	for small $ x $, $\exp x = 1 + x + \dots$ with few terms, no table	1 polynomial per binade group	1–2
---------------------	---	-------------------------------	-----

The choice: Tang. One proof, independent of the range. A different evaluator may be used where it makes the proof simpler, for instance the short Taylor series near 0, where no reduction is needed.

The code, eval_fix.c. It uses GMP's integers (mpz) and nothing else: no MPFR, no floating point beyond reading x and writing h, l, s . A fixed-point number is an integer Z standing for $Z/2^P$, with $P = 200$. The polynomial has degree 13, so that the accuracy is that of MPFR's 161 bits and the two instances can be compared candidate by candidate; degree 10 would be enough for $\kappa = 120$.

```
// The parameters of eval_fix.c.

#include <gmp.h>

#define FIX_P 200    // bits after the point of every fixed-point number
#define FIX_TAB 256  // table size: x = (256 k + j) ln2/256 + r
#define FIX_LG 32    // extra bits of ln2/256, so that N * ln2/256 is exact
                    // to 2^-FIX_P for |N| < 2^32
#define FIX_DEG 13   // degree of the Taylor polynomial of exp(r)

// Program 3, the proved evaluator: exp with integers only (GMP's mpz).
// Tang's scheme:
//   x = N ln2/256 + r, N = 256 k + j, 0 <= j < 256, |r| <= 2^-9.5
//   exp(x) = 2^k * 2^(j/256) * exp(r)
// A fixed-point number is an integer X standing for X / 2^FIX_P.
// Designed error (not yet proved): 2^-169 on y, 2^-158 after the split.

#include <assert.h>
#include "fix.h"
#include "eval.h"
#include "fix_table.h"

static mpz_t T[FIX_TAB];    // 2^(j/256), rounded to nearest
static mpz_t ln2;           // ln2/256, with FIX_LG extra bits
static mpz_t C[FIX_DEG + 1]; // 1/i!, rounded down
static mpz_t rmax;          // bound on |r|: 181/256 2^-9 < 2^-9.5
static mpz_t err;           // bound on the error of y: 2^-160

void
fix_init (void)
{
    for (int j = 0; j < FIX_TAB; j++)
        mpz_init_set_str (T[j], fix_T[j], 16);
    mpz_init_set_str (ln2, fix_ln2, 16);
    mpz_t f;
    mpz_init_set_ui (f, 1);
    for (int i = 0; i <= FIX_DEG; i++) {
        if (i > 0) mpz_mul_ui (f, f, i);
        mpz_init (C[i]);
        mpz_setbit (C[i], FIX_P);
        mpz_fdiv_q (C[i], C[i], f);
    }
    mpz_clear (f);
}
```

```

    mpz_init_set_ui (rmax, 181);
    mpz_mul_2exp (rmax, rmax, FIX_P - 17);
    mpz_init (err);
    mpz_setbit (err, FIX_P - 160);
}

// exp(x) = y / 2^FIX_P * 2^k; returns k
static long
fix_exp (mpz_t y, double x)
{
    mpz_t X, r, t;
    mpz_inits (X, r, t, NULL);
    // X = x 2^FIX_P, exactly
    int ex;
    double mx = frexp (x, &ex);
    long sh = ex - 53 + FIX_P;
    assert (sh >= 0);
    mpz_set_si (X, (long) ldexp (mx, 53));
    mpz_mul_2exp (X, X, sh);
    // r = x - N ln2/256, checked small
    long N = (long) nearbyint (x * (FIX_TAB / M_LN2));
    mpz_mul_si (t, ln2, N);
    mpz_fdiv_q_2exp (t, t, FIX_LG);
    mpz_sub (r, X, t);
    mpz_abs (t, r);
    assert (mpz_cmp (t, rmax) < 0);
    // exp(r) by Horner's rule, one truncation per product
    mpz_set (y, C[FIX_DEG]);
    for (int i = FIX_DEG - 1; i >= 0; i--) {
        mpz_mul (y, y, r);
        mpz_fdiv_q_2exp (y, y, FIX_P);
        mpz_add (y, y, C[i]);
    }
    // times 2^(j/256)
    long j = ((N % FIX_TAB) + FIX_TAB) % FIX_TAB;
    mpz_mul (y, y, T[j]);
    mpz_fdiv_q_2exp (y, y, FIX_P);
    // y is not within err of a power of 2, so its exponent is that of exp(x)
    mpz_sub (t, y, err);
    size_t nb = mpz_sizeinbase (t, 2);
    mpz_add (t, y, err);
    assert (mpz_sizeinbase (t, 2) == nb);
    mpz_clears (X, r, t, NULL);
    return (N - j) / FIX_TAB;
}

int
expo_fix (double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);
    int e = (int) mpz_sizeinbase (y, 2) - FIX_P + k;
    mpz_clear (y);
    return e;
}

```

```

// the top 53 bits of y, as a double scaled by 2^sc; y keeps the rest
static double
top53 (mpz_t y, long sc)
{
    if (mpz_sgn (y) == 0) return 0.0;
    long sh = (long) mpz_sizeinbase (y, 2) - 53;
    if (sh < 0) sh = 0;
    mpz_t hi;
    mpz_init (hi);
    mpz_fdiv_q_2exp (hi, y, sh);
    double d = ldexp (mpz_get_d (hi), sh + sc);
    mpz_mul_2exp (hi, hi, sh);
    mpz_sub (y, y, hi);
    mpz_clear (hi);
    return d;
}

void
eval_fix (double *h, double *l, double *s, double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);
    *h = top53 (y, k - FIX_P);
    *l = top53 (y, k - FIX_P);
    *s = top53 (y, k - FIX_P);
    mpz_clear (y);
}

// Is x hard to round at level m: is exp(x) 2^(54-e) within 2^-m of an
// integer? y has nb bits; its top 54 are the integer part, the other
// nb - 54 bits the fraction. Returns 1 (hard), 0 (not hard), or 2 when
// the error of y does not allow to decide; a caller that builds a
// superset keeps the undecided ones.
int
check_fix (double x, int m)
{
    mpz_t y, low, d, t;
    mpz_inits (y, low, d, t, NULL);
    fix_exp (y, x);
    long f = (long) mpz_sizeinbase (y, 2) - 54; // bits of the fraction
    mpz_fdiv_r_2exp (low, y, f); // the fraction, times 2^f
    mpz_set_ui (d, 0);
    mpz_setbit (d, f);
    mpz_sub (d, d, low); // distance to the next integer
    if (mpz_cmp (low, d) < 0) mpz_set (d, low); // d = the distance, times 2^f
    mpz_set_ui (t, 0);
    mpz_setbit (t, f - m); // the threshold 2^-m, times 2^f
    int r;
    mpz_add (low, d, err);
    if (mpz_cmp (low, t) < 0) r = 1;
    else {
        mpz_sub (low, d, err);
        r = mpz_cmp (low, t) >= 0 ? 0 : 2;
    }
}

```

```

mpz_clears (y, low, d, t, NULL);
return r;
}

```

The constants, the 256 values $2^{j/256}$ and $\ln 2/256$, are in `fix_table.h`, produced with MPFR by `gen_fix.c`. That producer is not trusted: each constant is certified in the proof. The coefficients $1/i!$ are computed exactly from integers and need no certificate.

`check_fix` computes the distance d of $Y(x)$ to the nearest integer from y , and answers 1 (hard) when $d + \text{err} < 2^{-m}$, 0 (not hard) when $d - \text{err} \geq 2^{-m}$, and 2 (undecided) otherwise.

5.2 The obligations on it

All four rest on one bound, on the fixed-point value y **before** it is split into doubles. `fix_exp` returns y and k with

$$(E) \quad |y \cdot 2^{k-P} - \exp x| \leq \text{err} \cdot 2^{k-P}, \quad \text{err} = 2^{P-160},$$

where `err` is the constant of the code. From (E):

#	obligation	from (E) and
3a	E1 and E2 for <code>eval_fix</code>	the split of y into h, l, s by <code>top53</code> truncates less than 2^{-158} relative, and each part is below the previous one's last bit
3b	E3 for <code>expo_fix</code>	the second <code>assert</code> of <code>fix_exp</code> : y is not within <code>err</code> of a power of 2, so y and $\exp x$ have the same exponent
3c	E4 for <code>check_fix</code> : it answers 0 only when x is not hard	the distance computation; answering 1 or 2 wrongly costs nothing

(E) is stated on y , not on $h + l + s$. The check reads y directly, so 3c does not suffer from the limit of three doubles below $x = -634$ (Section 6); 3a does.

5.3 How to prove it

The proof follows the computation of `fix_exp`, one step at a time. Each step is an integer operation with a floor, and each lemma bounds what it loses, in units of 2^{-P} (the numbers are **computed**, the proof is to do):

step	what to prove	how
$X = x \cdot 2^P$	exact	x is a 53-bit integer times a power of 2; the first <code>assert</code> in the code guarantees the shift is not negative
$r = X - \lfloor NL/2^{32} \rfloor$	within 2 units of $x - N \ln 2/256$	L is $\ln 2/256$ to $P + 32$ bits, certified by one <code>interval</code> call; $ N < 2^{19}$
$ r < 2^{-9.5}$	checked at run time	the <code>assert</code> on <code>rmax</code> ; the 2 units of slack stay under the bound
Horner	within a few units of $\sum_{i \leq 13} r^i/i!$	one floor per step, each damped by $ r < 2^{-9.5}$, so about 2 units; the $1/i!$ are floors of exact integers; the 2 units of r add about 2 more

Taylor	$ \exp r - \sum_{i \leq 13} r^i / i! \leq 2^{-169}$	one <code>interval</code> call over $ r \leq 2^{-9.5}$, like <code>cheb_valid</code> , or the remainder bound by hand
$\times T[j]$	within 1.5 units	each $T[j]$ is certified to half a unit: 256 <code>interval</code> calls at about 250 bits; one floor
sum	(E)	$2^{-169} + \text{a few units} \leq 2^{-160}$, with about 8 bits to spare

Then 3a, 3b and 3c are small lemmas on integers: the split into three 53-bit parts, the bit length of y , and the distance to the nearest integer.

In Rocq. The proof is about a model of `eval_fix.c` over $\mathbb{Z}$, with `Z.shiftr` for the floors, as in `Search.v`. The certificates are `interval` calls, as in `Cheb.v`. The difficulty is the one of the table below; the hardest part is the bookkeeping of the reduction.

part	difficulty
the Taylor bound on r : one <code>interval</code> call	1
the 256 table entries and $\ln 2/256$: one interval check each	1
the reduction, $ k \leq 1075$	3
the sum of the truncation errors, one per product	2–3
3a to 3c	1–2

6 The test

`make test` in `code/APaul/htrplan/` runs three programs on the same slice and compares their output line by line: every candidate, then the hard cases found.

- `htr_slice`: Program 1, `htr.c` itself, with its interval and m taken from the command line by a `sed`;
- `htrp mpfr`: Program 2 with Program 1's functions (Section 4);
- `htr3`: Program 2 with Program 3, `check_fix` included. It is linked without MPFR, so it cannot call it.

All numbers here are **measured**, 2026-09-28, on this desktop:

slice	chunks	candidates	result
$[0.25, 0.25 + 10^4 \cdot 2^{-34})$	10 000	410 325	identical in all three; one hard case, <code>0x1.00002385331bep-2</code> , the first of <code>htr.c</code> 's five; no undecided
$[-0.25 - 10^4 \cdot 2^{-34}, -0.25)$	10 000	498 078	identical in all three; one hard case; no undecided

Program 3 was also compared directly with MPFR at 400 bits on ten points between -700.3 and 700.1 : apart from the point below, its relative error is between 2^{-159} and 2^{-188} (**measured**). The h of the two evaluators is not always the same (Program 3 truncates where MPFR rounds); the contract does not ask it to be.

One limit found by the test. At $x = -700.3$ the error of $h + l + s$ is $2^{-66.5}$ for both evaluators (**measured**). There $\exp x \approx 2^{-1010}$, so l and s fall below the smallest double and are lost. Three doubles carry 159 bits only while $\exp x \geq 2^{-1074+159} = 2^{-915}$, that is $x \geq -634$ (**computed**). Below that the interface of Program 2 must change, for instance by passing the fixed-point value y instead of three doubles.

7 The number of candidates

About 41 per chunk on $[0.25, 0.25001)$ (**measured**: 7 056 503 over 171 799 chunks, `htr.c`'s own comment). The count comes from the width of the window, $2E/2^{64} \approx 2^{-14.6}$ per double (**computed**), which is almost all drift. A smaller n shrinks the drift quadratically and so the candidates, at the price of more chunks.

8 What to do, in order

1. **Program 2, 2a and 2b**, on $[0.25, 0.25001)$, where 2c and 2d are done. This settles the hard point, the window, for any evaluator that meets the contract.
2. **Generalise the search** to the whole range: the slope above 1, the cuts at binades, the subnormal and overflow ends, and an interface that passes y below $x = -634$.
3. **Program 3**: prove (E), then 3a to 3c, and certify the constants. The code is written and tested (Section 6). Add the short Taylor series near 0 if it simplifies things.
4. **The link to the running program**. The proofs above are about a Rocq model of the search. Tying them to the code that runs is a separate step; with Capla (our port of `htr_plain.c`, `code/APaul/capla/`) the program has a formal semantics, but how a Capla program is linked to a Rocq proof has not been tried.

A Annex: the initial program

Program 1 is `code/APaul/htr.c`, by Paul Zimmermann. It is not proved: it is the reference that Programs 2 and 3 are tested against. It uses MPFR in three places (**read**):

function	what it does	becomes in Program 2
<code>dd_exp</code>	exp x at 161 bits, split into three doubles $h + l + s$	<code>eval</code>
<code>ref_exp</code>	exp at both ends, rounded toward zero, to check e is the same on the interval	<code>expo</code>
<code>check</code>	decides a candidate, at $53 + m$ bits	the check in <code>report</code>

```
// search hard-to-round cases of exp in [0.25,0.5)
// with at least 43 identical bits after round bit

#include <stdio.h>
#include <stdint.h>
#include <mpfr.h>
#include <assert.h>
#include <math.h>

// reference code using MPFR, with RNDZ
double
ref_exp (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    double ret = mpfr_get_d (y, MPFR_RNDN);
    mpfr_clear (y);
    mpfr_set_emin (emin);
}
```

```

    return ret;
}

// put in h+l+s a triple-double approximation of exp(x)
static void
dd_exp (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

unsigned long checks = 0, found = 0;

static void
check (double x, int m)
{
    // printf ("check x=%la\n", x);
    checks ++;
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
    mpfr_set_d (y, x, MPFR_RNDN);
    mpfr_exp (z, y, MPFR_RNDN);
    mpfr_set (y, z, MPFR_RNDN);
    if (mpfr_cmp (y, z) == 0) {
        printf ("%la\n", x);
        found ++;
    }
    mpfr_clear (y);
    mpfr_clear (z);
}

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m0 identical bits after round bit
static void
search (double x0, double x1, int m)

```

```

{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

    // check ulp(x0) = ulp(nextbelow(x1))
    frexp (x0, &e0);
    frexp (nextafter (x1, x0), &e1);
    assert (e0 == e1);

    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
    double y0 = ref_exp (x0);
    double y1 = ref_exp (nextafter (x1, x0));
    frexp (y0, &e);
    frexp (y1, &e1);
    assert (e == e1);
    double uh = ldexp (1.0, e - 53); // uh = ulp(exp(x))

    double h, l, s;
    double x = x0;
    while (x < x1) {
        dd_exp (&h, &l, &s, x);
        double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
        double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

        // prepare loop
        // A is the bits of h+l after the round bit
        h = ldexp (h, 54 - e); // now ulp(h) = 2
        l = ldexp (l, 54 - e);
        // we now have -2 < l < 2
        if (l >= 1.0) l -= 1.0;
        while (l < 0) l += 1.0;
        assert (0 <= l && l < 1);
        s = ldexp (s, 54 - e);
        uint64_t lu = get_uint64 (l), su = get_uint64 (s);
        uint64_t A = lu + su;
        // scale the derivatives too
        dh = ldexp (dh, 54 - e);
        dl = ldexp (dl, 54 - e);
        dd = ldexp (dd, 54 - e);
        uint64_t B = get_uint64 (dh) + get_uint64 (dl);
        // the maximal error is bounded by dd/2*n^2
        dd = dd / 2.0 * n * n;
        // hard-to-round cases are at distance < 2^-(m+1) ulp(h)
        uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
        // we want to find values of A in [-E, E]
        A += E; // shift A
        // now we want to find values of A in [0, 2*E]
        for (uint64_t i = 0; i < n; i++) {
            if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
                double xi = x + i * ux;
                check (xi, m);
            }
            A += B;
        }
    }
}

```

```

    x += n * ux;
    if (x > x1)
        n = (x1 - x) / ux;
}
}

/* With x0 = 0.25, x1 = 0.25001, m = 35
on thym.loria.fr (Intel(R) Core(TM) Ultra 7 265) with gcc 15.3.0
(-O3 -march=native):

0x1.00002385331bep-2
0x1.0000561fc7e06p-2
0x1.0001cd08ef3d4p-2
0x1.00026c769e211p-2
0x1.00028a78a7303p-2
7056503 checks, found 5 hard-to-round

real 0m39.080s
user 0m39.032s
sys 0m0.008s

BaCSeL:
make GMP=$GMP MPFR=$MPFR FPLLL=$FPLLL DEFSAL="-DEXP -DBASIS=2 -DAUTOMATIC"
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -2 -nthreads 4
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t 18 -t0 4503599627370496 -t1
4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
Welcome to BaCSeL version 4.0, using 4 thread(s)
Compilation flags: -DBASIS=2 -DEXP -DAUTOMATIC
LLL parameters: delta=0.999 eta=0.501
Parameters for AUTOMATIC: FAILURE_LOWERBOUND=2.0 FAILURE_UPPERBOUND=4.0
*** x=0x4.0001587f1f818p-4, distance is: 1.059463030e-11 0x1.0000561fc7e06p-2
*** x=0x4.00008e14cc6f8p-4, distance is: 1.055287224e-11 0x1.00002385331bep-2
*** x=0x4.00073423bcf5p-4, distance is: 2.577853443e-12 0x1.0001cd08ef3d4p-2
*** x=0x4.000a29e29cc0cp-4, distance is: 7.860279955e-12 0x1.00028a78a7303p-2
*** x=0x4.0009b1da78844p-4, distance is: 7.913797859e-12 0x1.00026c769e211p-2
*/
int main()
{
    double x0 = 0.25, x1 = 0.25001;
    int m = 35;
    search (x0, x1, m);
    printf ("%lu checks, found %lu hard-to-round\n", checks, found);
}

```

How Program 2 gives it back is in Section 4.